

Cloud Computing and Virtualization

1

Lecture – 07

2

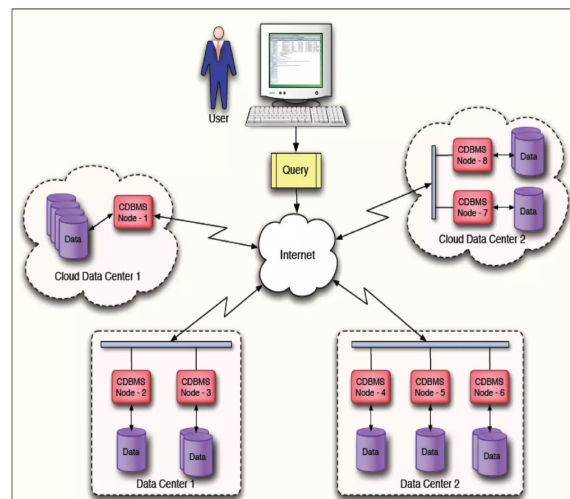
In this lecture...

- I. Storage in Cloud Computing
 - I. Database as a service
 - II. Storage model and file system
 - I. Parallel file system
 - II. Distributed file system

3

Cloud Storage Model

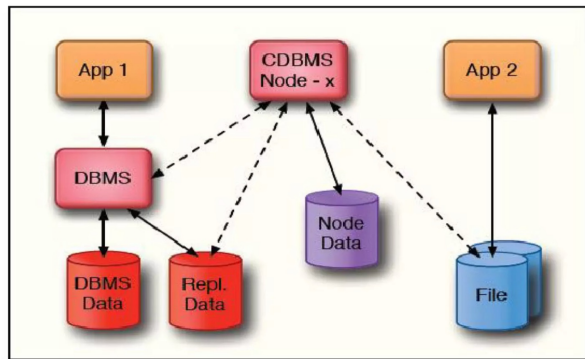
Cloud storage consists of multiple data centers and distributed nodes across different geographic locations that together store and manage database data, with each node holding metadata maps for efficient access.



4

Working of Node

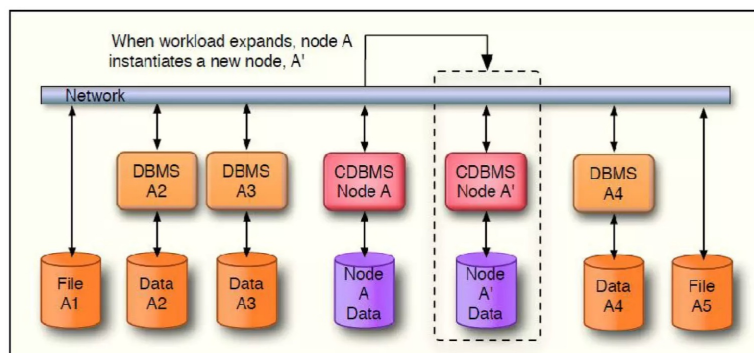
- A node in a cloud database receives a query, determines the best source for the required data, and then either retrieves it from the main database or from a replicate to respond to the user.
- A CDBMS node acts as an intelligent coordinator that processes application requests, locates data across databases, replicas, node storage, and files, and returns results transparently.



5

Splitting of a Node

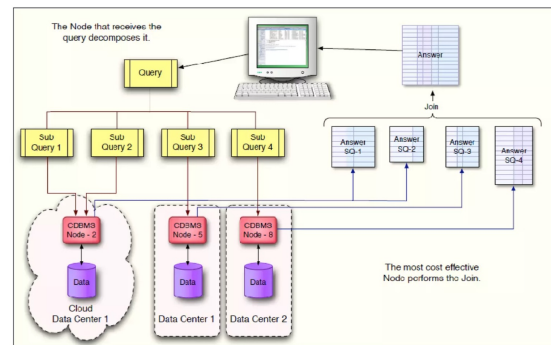
- When a node becomes overloaded with queries, the system creates a new node to share the workload, improving scalability and performance.



6

Query Splitting

- Distributed query processing splits a user's overall query into multiple sub-queries that are sent to different nodes, and the individual results are then combined and sent back to the user.



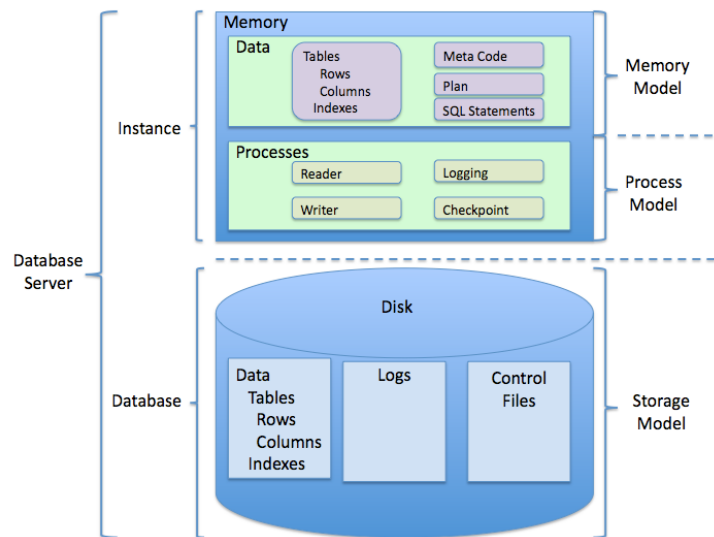
7

Database as a Service – DBaaS

- DBaaS is a cloud computing service model in which the cloud provider delivers a fully managed database to users.
- The user can create, store, query, and manage data without worrying about database installation, hardware, operating system, backups, or maintenance.
- DBaaS provides:
 - On-demand database provisioning
 - Automatic scaling of storage and performance
 - Built-in backup and recovery
 - High availability and fault tolerance
 - Pay-as-you-use pricing

8

Database as a Service – DBaaS



9

Choosing best DBaaS

- **Data Sizing:** The amount of data that the DBaaS can support (storage limits per database).
- **Portability:** Ability to move or access the database even if a provider goes out of business or in emergency situations.
- **Transaction Capabilities:** Support for transactional features, especially ACID transactions for reliable operations.
- **Configurability:** How easily the database configuration can be customized or managed by the user/administrator.
- **Database Accessibility:** Availability of standard interfaces and protocols (e.g., JDBC, REST, SOAP) for connecting to the database.
- **Certification and Accreditation:** Whether the provider has relevant security and compliance certifications (e.g., FISMA).
- **Data Integrity, Security, and Storage Location:** Strength of data protection mechanisms, encryption, and geographical storage considerations.

10

Challenges of DBaaS

- **Internet Dependency**
 - Performance depends heavily on network speed and availability.
- **Query Performance Variability**
 - Different types of queries (simple vs complex) may perform inconsistently.
 - Distributed execution adds overhead.
- **Multi-tenancy Resource Management**
 - Multiple users share the same infrastructure.
 - Difficult to guarantee equal performance for all tenants.
- **Scalability Management**
 - Although DBaaS supports elasticity, managing dynamic scaling efficiently is complex.
- **Security and Privacy Concerns**
 - Data is stored off-premises.
 - Risk of unauthorized access or data leakage.

11

Advantages of DBaaS

- **Reduced Administrative Overhead**
 - No need to install, configure, or maintain databases.
 - Provider handles updates, backups, and patches.
- **Elastic Scalability**
 - Storage and compute resources can be scaled up or down automatically.
- **High Availability**
 - Replication and fault tolerance ensure minimal downtime.
- **Cost Effectiveness**
 - Pay-as-you-use pricing model and no upfront hardware investment.
- **Faster Deployment**
 - Databases can be created and used within minutes.
- **Automatic Backup and Recovery**
 - Built-in mechanisms protect against data loss.

12

Limitations of DBaaS

- **Limited Control**
 - Users have restricted access to database configuration and tuning.
- **Vendor Lock-in**
 - Migrating data from one DBaaS provider to another can be difficult.
- **Security Trust Issues**
 - Users must trust the provider to protect sensitive data.
- **Cost Over Time**
 - Long-term usage may become expensive for data-intensive applications.
- **Customization Restrictions**
 - Not all database features or extensions may be supported.

13

Cloud File Systems

- A cloud file system provides a hierarchical structure (files and folders) similar to traditional operating system file systems (like NTFS), but the data is physically stored in cloud data centers.
- Examples: Google Filesystem, HDFS
- A cloud file system lets you store and manage files on the internet with high reliability, scalability, and global access, making it ideal for modern cloud-based applications.

14

Distributed File System

- A Distributed File System (DFS) in cloud computing is a file system that stores files across multiple servers (nodes) in a distributed manner while making them appear to users and applications as a single unified file system.
- The main goal of DFS is to provide:
 - Scalability
 - Fault tolerance
 - High availability
- Example: HDFS (Hadoop Distributed File System), Google File System (GFS)
- Traditional file systems are limited to a single machine and fail if the disk or system crashes but DFS handles huge volumes of data, continues to work even if some nodes fail.

15

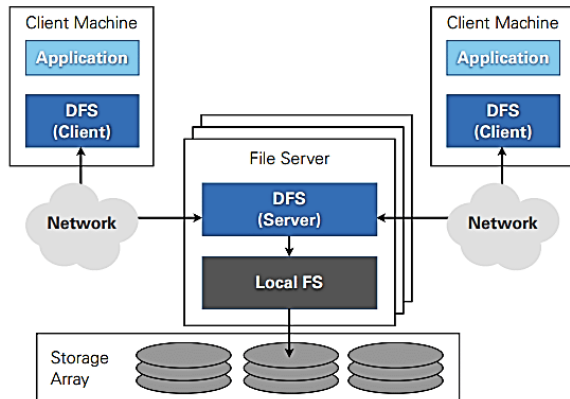
Components of Distributed File System

- **Client**
 - User or application accessing files
- **Metadata Server**
 - Manages file system namespace
 - Stores metadata (file name, permissions, block locations)
- **Data Nodes / Storage Nodes**
 - Store actual file blocks
- **Network**
 - Connects all nodes
- **Replication Manager**
 - Ensures multiple copies of data for fault tolerance

16

Working Mechanism of Distributed File System

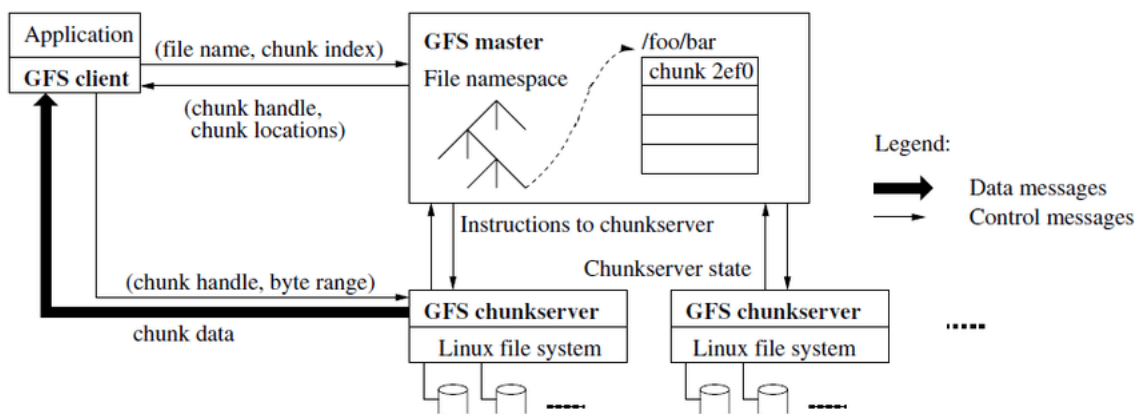
- **Client Request**
 - Client requests to read or write a file
- **Metadata Lookup**
 - Client contacts the Metadata Server
 - Metadata server returns block locations
- **Data Access**
 - Client directly accesses multiple Data Nodes
- **Replication**
 - Each data block is stored on multiple nodes
- **Failure Handling**
 - If a node fails, data is accessed from replica



17

Case Study

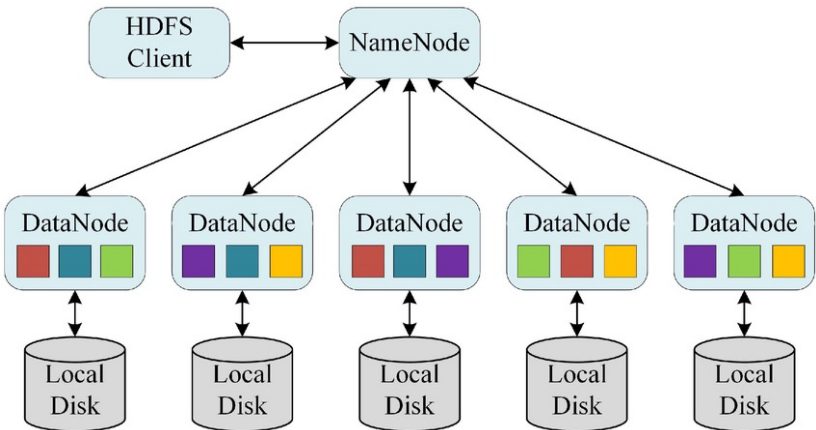
- Google file system



18

Case Study

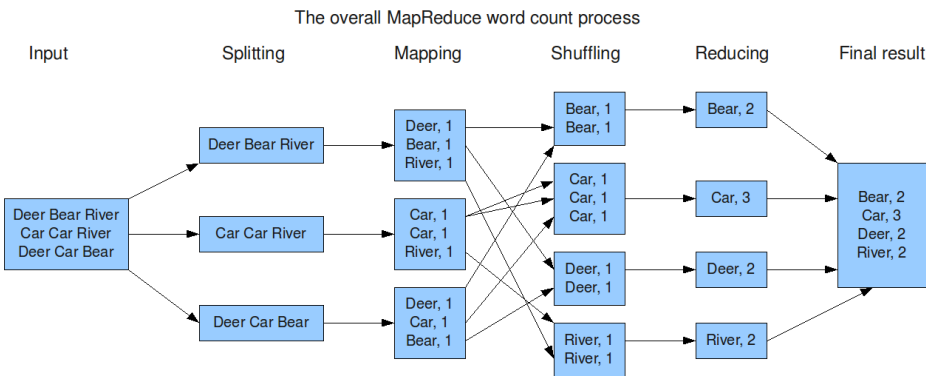
- Hadoop File System - HDFS



19

Case Study

- Map-Reduce



20

Parallel File System

- A Parallel File System (PFS) in cloud computing is a high-performance distributed file system that allows multiple clients (VMs, containers, compute nodes) to read and write different parts of the same file simultaneously.
- It is designed to handle large data sets and high I/O throughput, commonly used in HPC, big data analytics, AI/ML, and scientific computing in the cloud.
- Distributed File System focuses on distribution, replication, and reliability. Where parallel File System builds on distributed concepts and adds parallelism at file-access level.
- Traditional file systems allow limited concurrent access that become bottlenecks for large files where parallel file systems split files into chunks, distribute chunks across multiple storage servers and enable simultaneous access, increasing performance.

21

Components of Parallel File System

- Client Nodes
 - Cloud VMs or containers running applications
- Metadata Server (MDS)
 - Manages file metadata (name, permissions, location)
- Object Storage Servers (OSS) / Data Servers
 - Store actual file data in chunks
- High-Speed Network
 - Enables fast communication between clients and servers
- Storage Devices
 - SSDs / HDDs / NVMe in distributed data centers

22

Parallelism in Distributed File System

Parallel File Systems are built on top of distributed file systems, but they allow multiple clients to read/write different parts of the same file simultaneously.

1. File Striping (Data Chunking)
- In PFS, each file is divided into stripes/chunks. Each stripe is stored on different storage servers (data nodes). Clients can access different stripes at the same time.
2. Dedicated Metadata Server
- PFS uses metadata servers (MDS) to track which server holds which stripe, File layout, permissions, and offsets
 - Clients query the metadata once to get layout and then access servers in parallel, avoiding sequential access bottleneck present in typical DFS.
3. Parallel I/O Operations
- Read/Write requests are split across multiple storage servers.
 - Each server handles I/O independently, increasing throughput.
 - DFS often requires one node to serve sequential block requests, limiting speed.

23

Distributed vs Parallel File System

DFS	PFS
To store files across multiple machines and provide a single unified view	To provide high-performance, parallel access to the same file
Primary focus is Scalability, fault tolerance, availability	Primary focus is High throughput and low latency I/O
Typically one client accesses a file at a time	Multiple clients access different parts of the same file simultaneously
File is split into blocks stored on different nodes	File is striped into chunks accessed in parallel
Matadata is managed through centralized or semi-centralized metadata server	Dedicated metadata servers, often clustered
Example: HDFS, GFS	Example: Lustre, GPFS

24

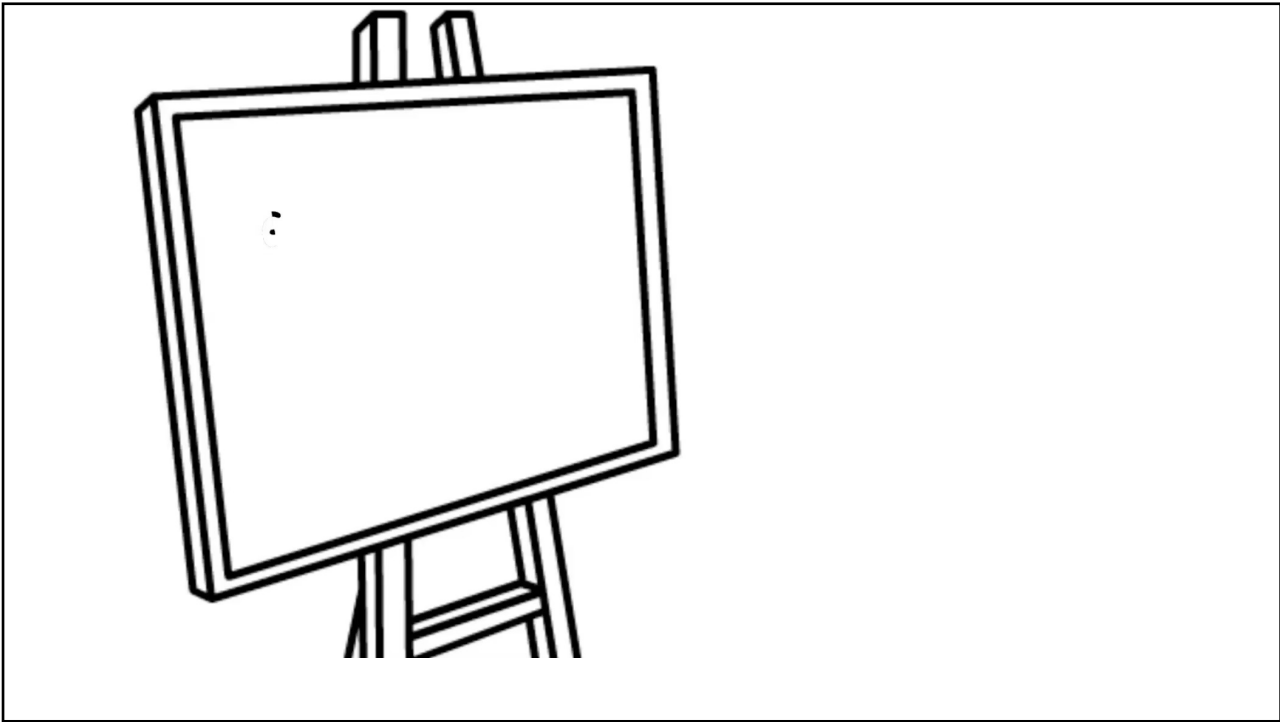
Lecture – 08

25

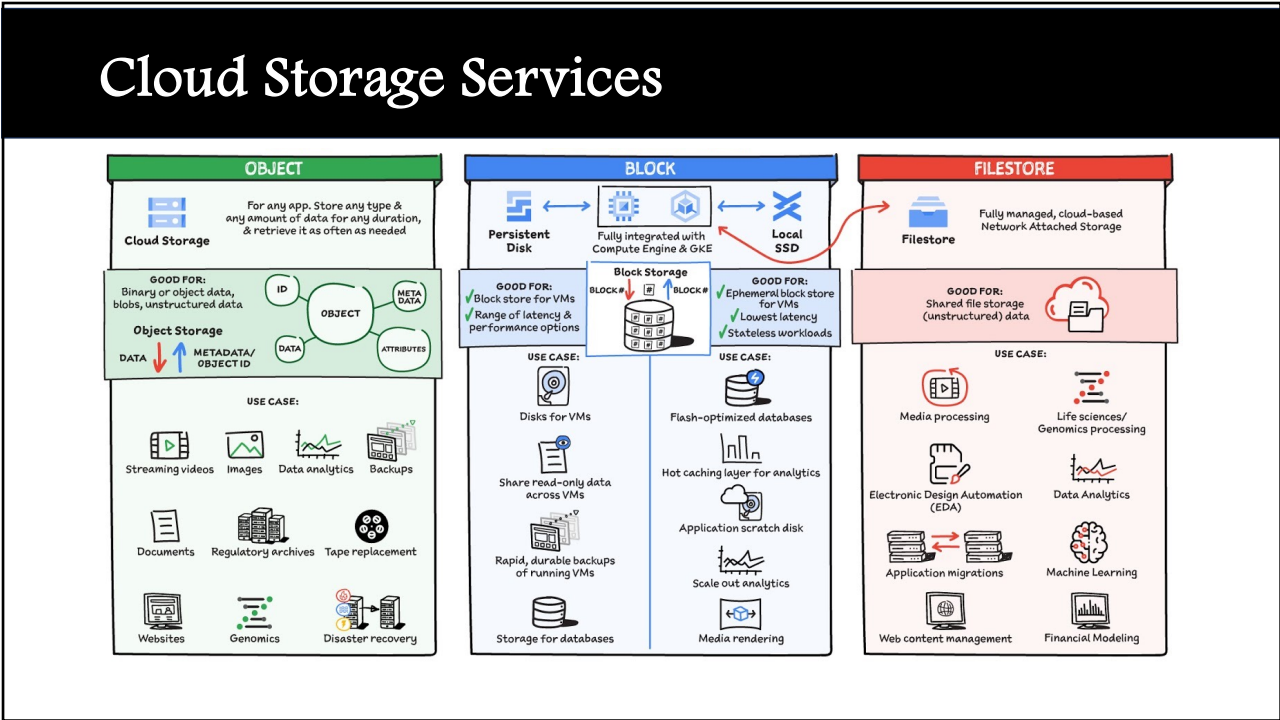
In this lecture...

- I. Cloud Storage Services
 - I. Object storage
 - II. File storage
 - III. Block storage
 - IV. Archive storage
- II. Elastic storage and data redundancy

26



27



28

Object Storage

- Object Storage is a cloud storage method that stores large amounts of unstructured data such as logs, media files, sensor data, and web content by saving data as individual objects. Each object includes the data itself, custom metadata, and a unique identifier.
- Key Concepts
 - Objects: Units of stored data (any format, any size).
 - Metadata: Flexible descriptive information attached to each object, enabling rich search and retrieval.
 - Unique Identifiers: Every object gets a distinct name or ID for lookup.
- No traditional folder hierarchy is used instead objects are stored together in a flat storage pool called a data lake. Retrieval is done via metadata or identifier searches, not by navigating directory paths.

29

Object Storage

- Benefits of Object Storage
 - Highly scalable: effectively unlimited capacity.
 - Flexible metadata: rich indexing for retrieval.
 - Cost-effective: pay only for used storage.
 - Resilient & distributed: cloud providers replicate objects for durability and availability.
- Object storage is especially useful for:
 - Large media files (videos, images)
 - Application assets and backups
 - Logs, analytics data, machine learning datasets
 - Data archiving

30

Block Storage

- Block storage in cloud storage stores data in fixed-size blocks, and each block is treated as an independent unit. These blocks are attached to a virtual machine (VM) as a raw disk, and the operating system manages the file system on top of it.
- How block storage stores data
 - Data is split into equal-sized blocks
 - Each block has a unique address
 - Blocks are stored independently
 - VM accesses blocks directly using addresses
 - OS handles: File system, Formatting and Permissions
- Key Characteristics
 - No metadata or hierarchy (handled by OS)
 - Low latency access
 - Single VM access
 - Persistent storage

31

Block Storage

- Benefits of Block Storage
 - High performance and speed
 - Low latency
 - Fine control over storage
 - Reliable for transactional data
 - Supports snapshots and backups
 - Scales easily with cloud resources
- Use Cases of Block Storage
 - Databases (MySQL, Oracle, PostgreSQL)
 - Virtual machine boot volumes
 - Enterprise applications (ERP, CRM)
 - Transaction-based systems
 - High-performance workloads

32

File Storage

- File storage is a cloud storage method where data is stored as files organized in a hierarchical directory structure and accessed over a network using standard file protocols.
- How File Storage Stores Data
 - Data is stored as files
 - Files are arranged in folders and subfolders
 - Each file has: Name, Path and Permissions
 - Accessed using: NFS (Linux/Unix) SMB/CIFS (Windows)
- Key Characteristics
 - Hierarchical structure (folders)
 - Supports shared access
 - File locking & permissions
 - Persistent storage
 - Moderate performance

33

File Storage

- Benefits of File Storage
 - Simple and familiar usage
 - Easy file sharing
 - Multi-user access
 - Access control & security
 - Scalable storage capacity
- Use Cases of File Storage
 - Shared folders and home directories
 - Content management systems (CMS)
 - Enterprise file sharing
 - Web applications
 - Legacy applications

34

Archive Storage

- Archive storage is a cloud storage service designed for long-term data retention where data is rarely accessed. It offers very low storage cost but slower retrieval time compared to other storage types.
- How Archive Storage Stores Data
 - Data is stored as objects (similar to object storage).
 - Objects are moved to an archive tier after they are no longer frequently used.
 - Data is replicated across multiple locations for durability.
 - Data is not immediately accessible: Retrieval may take minutes to hours
 - Access is usually through APIs, not direct mounting.
- Archive Storage Services
 - AWS – S3 Glacier / Glacier Deep Archive
 - Microsoft Azure – Azure Archive Storage
 - Google Cloud – Archive Storage

35

Archive Storage

- Benefits of Archive Storage
 - Lowest storage cost
 - High durability (often 99%)
 - Ideal for long-term retention
 - Automatic lifecycle management
 - Secure and compliant (audit & retention policies)
- Use Cases of Archive Storage
 - Compliance and regulatory data
 - Legal records and audit logs
 - Long-term backups
 - Historical data and research data
 - Disaster recovery archives

36

Archive Storage

- Benefits of Archive Storage
 - Lowest storage cost
 - High durability (often 99%)
 - Ideal for long-term retention
 - Automatic lifecycle management
 - Secure and compliant (audit & retention policies)
- Use Cases of Archive Storage
 - Compliance and regulatory data
 - Legal records and audit logs
 - Long-term backups
 - Historical data and research data
 - Disaster recovery archives

37

Elastic Cloud Storage

- Elastic cloud storage is cloud-based data storage as a service that's able to expand and contract according to need and is billed on a pay-as-you-go model.
- It provides on-demand storage capacity and adaptability to changing workloads and supports various data types and protocols, making it suitable for a wide range of applications and industries.
- Characteristics of Elastic Cloud Storage:
 - Virtualization of storage - Physical disks are abstracted into virtual storage pools.
 - Storage pooling - Multiple disks across data centers act as one logical storage system.
 - Automatic provisioning - When data increases, new storage blocks/objects are allocated dynamically.
 - Auto-scaling - No fixed size limit; capacity expands transparently.
 - Pay-as-you-go - Users are billed only for storage consumed.

38

Elastic Cloud Storage

- Benefits of Elastic Storage
 - No upfront capacity planning
 - Handles unpredictable workloads
 - Cost-efficient (pay only for usage)
 - High availability and performance
 - Ideal for cloud-native applications
- Use Cases of Elastic Storage
 - Web applications with variable traffic
 - Big data analytics
 - Backup and recovery systems
 - Media streaming platforms
 - AI/ML datasets

39

Data Redundancy in Cloud Storage

- Data redundancy is the practice of storing multiple copies of data across different disks, servers, or geographic locations to ensure data durability, fault tolerance, and availability.
- **Elasticity handles growth; redundancy handles failure.**
- Data redundancy approaches:
 1. Replication
 - Exact copies of data are stored.
 2. Erasure Coding
 - Data is split into fragments + parity blocks.
 - Data can be reconstructed even if some fragments are lost.
 - More storage-efficient than replication.

40

Data Redundancy in Cloud Storage

Types of Data Redundancy

- a. Local Redundancy
 - Copies stored within the same data center.
- b. Zone Redundancy
 - Copies stored across different availability zones.
- c. Geographic Redundancy
 - Copies stored in different regions or countries.

41

Lecture – 09

42

In this lecture...

- I. Databases in cloud
 - I. Relational – SQL
 - II. Non-Relational – NoSQL
 - III. Graph Database

43

Databases in Cloud

A cloud database is not just a database hosted online, it is a managed service that handles scaling, availability, backup, and security for customers.

On-Prem Database	Cloud Database
Manual installation	API based installation
Vertical scaling only	Horizontal + Vertical
Backup is taken manually	Automated backup system
Single server	Multiple servers in multiple regions
Fixed hardware	Pay-as-you-go

44

Relational Database

- A relational database stores data in tables (rows & columns) and uses SQL for querying.
- Key properties
 - Schema-based (fixed structure)
 - ACID properties
 - Atomicity – Transaction completes fully or not at all (bank transfer)
 - Consistency – Rules are always followed
 - Isolation – Multiple users don't interfere
 - Durability – Data survives crashes
 - Strong consistency
 - Best for structured data

45

Relational Database

Cloud doesn't change SQL but it changes how SQL is managed.

- Query will be same as:
`SELECT * FROM students WHERE cgpa > 3.5;`
- But cloud handles:
 - Replication
 - Automatic backups
 - Failover
 - Scaling read replicas

46

Non Relational Database – NoSQL

- Non-relational databases, also known as NoSQL databases, are designed for handling unstructured or semi-structured data with flexible schemas.
- NoSQL (Not Only SQL) databases are designed to handle:
 - Huge volumes of data
 - High-speed read/write operations
 - Flexible or unstructured data
 - Horizontal scaling (adding more machines easily)

47

Non Relational Database – NoSQL

Key Characteristics of Cloud NoSQL databases

- **Schema-less Model:** Each record can have different fields

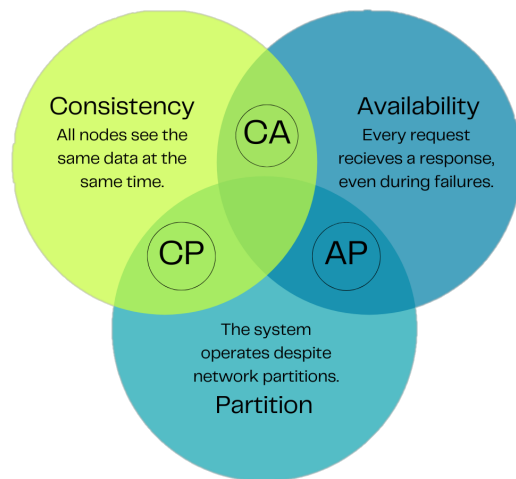
```
{ "id":1, "name": "Ram", "age":21 }
{ "id":2, "name": "Sita", "address": "Kathmandu" }
```
- **Horizontal Scalability:** Scale by adding more servers, not upgrading one machine
- **High Availability**
 - Data replicated across regions
 - System stays online even if one server fails
- **Eventual Consistency (CAP Theorem)**
 - Data may take time to sync across nodes
 - Prioritizes availability + partition tolerance

48

Non Relational Database – NoSQL

CAP Theorem

- CAP Theorem states that in a distributed database system, it is impossible to guarantee Consistency, Availability and Partition Tolerance at the same time.
- You can only fully achieve any two out of the three.



49

SQL vs NoSQL Databases

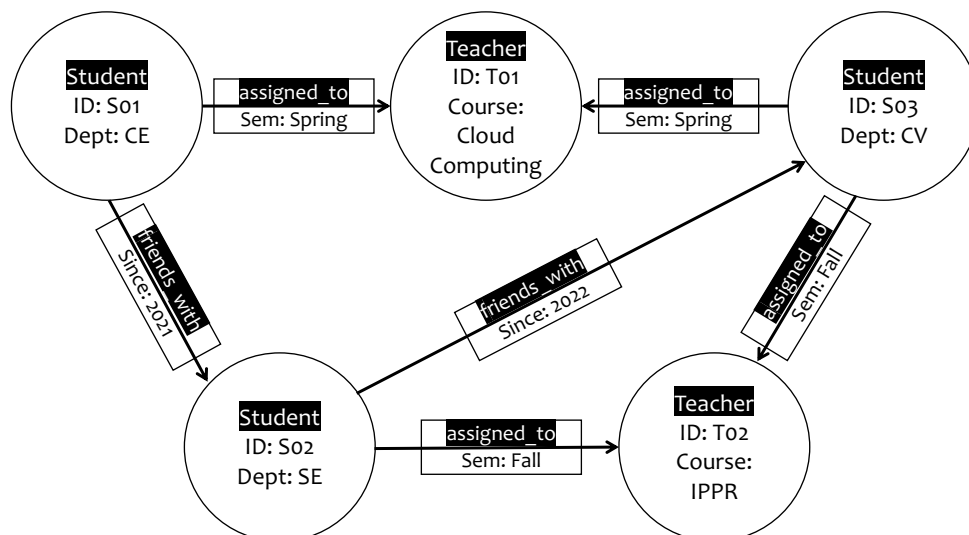
Relational (SQL) Databases	Non-Relational (NoSQL) Databases
Data are stored in tables with fixed rows/columns and relationships	Flexibility in data storage: documents (JSON-like), key-value, graphs, wide-columns
Fixed/predefined schema	Dynamic/schema-less
Standardized SQL (joins, complex queries)	Varies (e.g., MongoDB Query Language, APIs)
Primarily uses vertical scalability (upgrade hardware); cloud enables some horizontal	Uses horizontal scalability (add servers/nodes); ideal for massive/distributed data
Follows ACID properties.(strong, immediate consistency)	Follows CAP theorem and provide BASE (eventual consistency for availability)
Structured data (e.g., transactions, records)	Un/semi-structured (logs, JSON, multimedia)
Best for complex queries, data integrity, transactions	Best for high-volume, real-time, evolving data models

50

Graph Database

- A Graph Database is a type of NoSQL database that stores data in the form of nodes, edges, and properties.
- Components of a Graph Database:
 - **Nodes:** Represent entities - Eg. Person, Student, Movie, City
`User {name: 'Radheshyam', age: 25}`
 - **Edges (Relationships):** Define how nodes are connected, always directional and they have some meaning.
`Radheshyam) —[FRIENDS_WITH]→ (Phulkumari)`
 - **Properties:** Key-value pairs stored in nodes or edges.
`FRIENDS_WITH {since: 2021}`

51

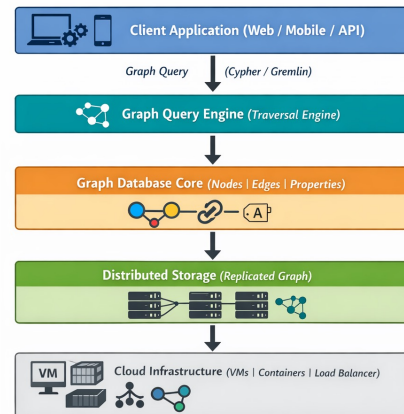


52

Graph Database

- Graph databases store direct pointers between nodes. No expensive JOIN operations and relationships are stored physically with data.
- Queries are executed using graph traversal: DFS, BFS, Pattern matching
- Graph databases use graph-specific query languages: Neo4j → Cypher

```
MATCH (u:User)-[:FRIEND_OF]->(f)
RETURN f.name
```



53

Graph Database – Advantages

- Performance
 - Relationship queries run in constant time
 - Faster than JOIN-heavy SQL queries
- Scalability
 - Horizontal scaling in cloud
 - Handles billions of nodes & edges
- Flexibility
 - Schema-optional
 - Easy to add new relationships

54

Graph Database – Use Cases

- Social Networks (Facebook, LinkedIn)
 - Friend recommendations
 - Mutual connections
 - Community detection
- Recommendation Systems (Netflix, Amazon)
 - “Users like you also watched...”
 - Personalized suggestions
- Knowledge Graphs (Google Search)
 - Semantic search
 - Question answering systems
- Network & IT Infrastructure (Cloud networking)
 - Impact analysis
 - Fault detection
 - Dependency mapping